EXVUL

SMART CONTRACT AUDIT REPORT

O1 Launch Tax Contracts Smart Contract

SEPTEMBER 2026

Contents

1. EXECUTIVE SUMMARY **4**
1.1 Methodology 4

2. FINDINGS OVERVIEW **7**
2.1 Project Info And Contract Address 7
2.2 Summary 7
2.3 Key Findings 8

3. DETAILED DESCRIPTION OF FINDINGS **9**
3.1 Pending dividends can be captured by a temporary holder 9
3.2 Dividends paid to an unexcluded V2 pair can be stolen with skim 12
3.3 Tick-spacing and band updates can make registered quotes unlaunchable 15
3.4 A nonpayable platform recipient can disable launches when the native launch fee is nonzero 17
3.5 Native dividends trust an immutable wrapped-native dependency by address and code presence only 18
3.6 Metadata is unbounded and unnormalized 20
3.7 Burn delivery is deferred, contrary to the supplied PR description 22
3.8 The supplied validation snapshot is stale relative to the final source 24

4. CONCLUSION **25**

5. APPENDIX **26**
5.1 Basic Coding Assessment 26
5.1.1 Apply Verification Control 26
5.1.2 Authorization Access Control 26
5.1.3 Forged Transfer Vulnerability 26
5.1.4 Transaction Rollback Attack 27
5.1.5 Transaction Block Stuffing Attack 27
5.1.6 Soft Fail Attack Assessment 27
5.1.7 Hard Fail Attack Assessment 28
5.1.8 Abnormal Memo Assessment 28
5.1.9 Abnormal Resource Consumption 28
5.1.10 Random Number Security 29
5.2 Advanced Code Scrutiny 29
5.2.1 Cryptography Security 29
5.2.2 Account Permission Control 29

5.2.3 Malicious Code Behavior 30
5.2.4 Sensitive Information Disclosure 30
5.2.5 System API 30

6. DISCLAIMER **31**

7. REFERENCES **32**

8. About Exvul Security **33**

1. EXECUTIVE SUMMARY

ExVul Web3 Security was engaged by **O1 Launch Tax Contracts** to review smart contract implementation. The assessment was conducted in accordance with our systematic approach to evaluate potential security issues based upon customer requirement. The report provides detailed recommendations to resolve the issue and provide additional suggestions or recommendations for improvement.

The outcome of the assessment outlined in chapter 3 provides the system's owners a full description of the vulnerabilities identified, the associated risk rating for each vulnerability, and detailed recommendations that will resolve the underlying technical issue.

1.1 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [10] which is the gold standard in risk assessment using the following risk models:

- **Likelihood**: represents how likely a particular vulnerability is to be uncovered and exploited in the wild.
- **Impact**: measures the technical loss and business damage of a successful attack.
- **Severity**: determine the overall criticality of the risk.

Likelihood can be: High, Medium and Low and impact are categorized into: High, Medium, Low, Informational. Severity is determined by likelihood and impact and can be classified into five categories accordingly: Critical, High, Medium, Low, Informational shown in table 1.1.

Likelihood \ IMPACT	Informational	Low	Medium	High
High	INFO	MEDIUM	HIGH	CRITICAL
Medium	INFO	LOW	MEDIUM	HIGH
Low	INFO	LOW	LOW	MEDIUM

Table 1.1 Overall Risk Severity

To evaluate the risk, we will be going through a list of items, and each would be labelled with a severity category. The audit was performed with a systematic approach guided by a comprehensive assessment list carefully designed to identify known and impactful security issues. If our tool or analysis does not identify any issue, the contract can be considered safe regarding the assessed item. For any discovered issue, we might further deploy contracts on our private test environment and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.2.

- **Basic Coding Bugs**: We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- **Code and business security testing**: We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- **Additional Recommendations**: We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

Category	Assessment Item
Basic Coding Assessment	• Apply Verification Control • Authorization Access Control • Forged Transfer Vulnerability • Forged Transfer Notification • Numeric Overflow • Transaction Rollback Attack • Transaction Block Stuffing Attack • Soft Fail Attack • Hard Fail Attack • Abnormal Memo • Abnormal Resource Consumption • Secure Random Number

Advanced Source Code Scrutiny	• Asset Security • Cryptography Security • Business Logic Review • Source Code Functional Verification • Account Authorization Control • Sensitive Information Disclosure • Circuit Breaker • Blacklist Control • System API Call Analysis • Contract Deployment Consistency Check • Abnormal Resource Consumption
Additional Recommendations	• Semantic Consistency Checks • Following Other Best Practices

Table 1.2: The Full List of Assessment Items

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [14], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development.

2. FINDINGS OVERVIEW

2.1 Project Info And Contract Address

Project Name	Audit Time	Language
O1 Launch Tax Contracts	10/09/2026 - 15/09/2026	Solidity

Repository
https://github.com/o1exchange/o1-launch

Commit Hash
7a9697c4e8197b1e73e1ac8a250860336731814d

2.2 Summary

Severity	Found
CRITICAL	0
HIGH	1
MEDIUM	1
LOW	3
INFO	3



2.3 Key Findings

Severity	Findings Title	Status
HIGH	Pending dividends can be captured by a temporary holder	Fixed
MEDIUM	Dividends paid to an unexcluded V2 pair can be stolen with skim	Fixed
LOW	Tick-spacing and band updates can make registered quotes unlaunchable	Fixed
LOW	A nonpayable platform recipient can disable launches when the native launch fee is nonzero	Fixed
LOW	Native dividends trust an immutable wrapped-native dependency by address and code presence only	Fixed
INFO	Metadata is unbounded and unnormalized	Fixed
INFO	Burn delivery is deferred, contrary to the supplied PR description	Fixed
INFO	The supplied validation snapshot is stale relative to the final source	Fixed

Table 2.3: Key Audit Findings

3. DETAILED DESCRIPTION OF FINDINGS

3.1 Pending dividends can be captured by a temporary holder

SEVERITY: HIGH **STATUS:** Fixed

PATH:

`contracts/src/tax/TaxLaunchToken.sol`

`contracts/src/tax/TaxDividend.sol`

DESCRIPTION:

Dividend fees are accrued during swaps, but they are not assigned to the holders that existed when those fees were generated. Assignment happens only when any caller later invokes `allocatePendingDividends` or `allocateAndWithdrawDividendsFor`.

Every token transfer immediately replaces each participant's Dividend share with its current balance:

```
// TaxLaunchToken.sol:171-186
function _transfer(address from, address to, uint256 amount) internal {
    ...
    _balances[from] = bal - amount;
    _balances[to] += amount;
    ...
    if (tracker != address(0)) {
        ITaxDividend(tracker).setShare(from, _balances[from]);
        if (to != from) ITaxDividend(tracker).setShare(to, _balances[to]);
    }
}
```

Any caller can then allocate the complete pending balance across those current shares and immediately claim for a selected holder:

```
// TaxDividend.sol:236-247, 324-339
function allocateAndWithdrawDividendsFor(address holder, bool
    unwrapWrappedNative)
     external
     nonReentrant
     returns (uint256 amountAllocated, bool paid)
{
     amountAllocated = _allocatePendingDividends();
     paid = _withdrawDividendOfUser(holder, unwrapWrappedNative);
}

function _allocatePendingDividends() internal returns (uint256
    amountAllocated) {
     _requireSettled();
     uint256 pending = pendingDividendFees();
     if (pending == 0 || totalShares == 0) return 0;
     ...
     amountAllocated = Math.min(pending, balance - backing);
     totalDividendFeesAllocated += amountAllocated;
     _allocate(amountAllocated);
}
```

`_requireSettled()` only blocks allocation during one active PoolManager unlock. It does not prevent an attacker contract from using three consecutive unlocks inside one outer transaction:

1. Buy launch tokens and immediately become an eligible holder.
2. Redeem and allocate the historical pending fees, then claim the attacker's new entitlement.
3. Sell the temporary token position.

If pending rewards are `D`, incumbent eligible shares are `H`, attacker shares are `x`, and total round-trip cost is `L(x)`, the attack is profitable whenever:

```
D * x / (H + x) > L(x)
```

When `H == 0`, the first qualifying buyer receives almost the entire backlog.

This is the mechanically exploitable consequence of the documented funding-time allocation policy: "funding time" is the later permissionless allocation operation, not the earlier trades that accrued the fees.

IMPACT:

Verified by isolated PoC against the exact PR head and real Hook, FeeEscrow, Dividend and PoolManager accounting:

```
Pending dividend fees: 379.999999999999999980 quote
Attacker starting capital: 100 quote
Attacker ending balance: 470.999999999999999979 quote
Net profit: 370.999999999999999979 quote
```

The attacker paid the configured 10% buy and sell taxes and returned through the same pool. A second case retained a persistent incumbent holder: the attacker still earned `336.454542255454574231` quote while the incumbent received only `35.545457744545425748` from a `380.999999999999999980` quote backlog.

A real executor completed buy, allocation, claim and sell through one outer call. No owner, creator, keeper, victim approval, direct storage modification or fabricated FeeEscrow credit was used. The amount at risk is every unallocated Dividend fee, which can accumulate without a protocol cap.

The measured attack segment consumed 636,176 gas, leaving a wide profit margin in the reproduced case.

RECOMMENDATIONS:

Checkpoint Dividend entitlements when fees accrue, using the eligible share state that existed before a new buyer receives tokens. If physical FeeEscrow redemption must remain deferred, defer only funding, not entitlement assignment.

Previous-block snapshots, TWAB accounting or a holding-age requirement are alternatives. Define an explicit non-capturable destination or time-weighted carry rule for fees generated while nobody is eligible. A keeper that allocates frequently reduces the backlog but does not close the atomic path.

3.2 Dividends paid to an unexcluded V2 pair can be stolen with skim

SEVERITY: MEDIUM **STATUS:** Fixed

PATH:

`contracts/src/tax/TaxDividend.sol`

`contracts/src/tax/TaxLaunchToken.sol`

DESCRIPTION:

Every nonexcluded address holding at least the fixed `minimumDividendHolding` becomes a Dividend shareholder. Mandatory exclusions contain only a static set of system addresses established when the clone is initialized:

```
// TaxDividend.sol:136-150
_excludeMandatory(address(0));
_excludeMandatory(BURN_SINK);
_excludeMandatory(address(this));
_excludeMandatory(taxToken);
_excludeMandatory(_owner);
_excludeMandatory(hook);
_excludeMandatory(address(feeEscrow));
_excludeMandatory(address(poolManager));
for (uint256 i; i < configuration.defaultExclusions.length; ++i) {
    ...
}
```

A later-created V2-style TaxLaunchToken/quote pair is not automatically excluded. Transfers into the pair immediately give the pair a Dividend share through `TaxLaunchToken._transfer`.

Claims are permissionless and the default receiver is the holder itself:

```
// TaxDividend.sol:208-211, 279-281
function receiverOf(address holder) public view returns (address receiver)
    {
    receiver = _dividendReceiver[holder];
    if (receiver == address(0)) receiver = holder;
}

function withdrawDividendsFor(address holder, bool unwrapWrappedNative)
    external
    nonReentrant
    returns (bool paid)
{
    return _withdrawDividendOfUser(holder, unwrapWrappedNative);
}
```

The complete preconditions are a dividend-enabled launch, a secondary V2-style pair holding at least the fixed minimum, no exclusion or safe receiver override for the pair, and the Dividend reward quote being the pair's other asset. Under those conditions, an attacker can:

1. Call `allocateAndWithdrawDividendsFor(pair, false)`, or call `withdrawDividendsFor(pair, false)` after allocation.
2. Cause the Dividend contract to transfer quote tokens directly to the pair.
3. Leave the pair's quote balance greater than its recorded reserve.
4. Call the V2 pair's permissionless `skim(attacker)` and receive the excess.

The attacker does not need LP tokens, launch tokens or any privileged role.

IMPACT:

Every quote dividend attributed to the secondary pool's launch-token reserves can be stolen by the first caller that claims and skims it. The attack can repeat after every new allocation until the pair is excluded or assigned a safe receiver.

Verified by an isolated zero-capital PoC using a real tax-suite launch, real Dividend clone, ordinary Hook trades, FeeEscrow-backed fees and a V2-faithful pair:

```
Pending dividend fees: 380.999999999999999980 quote
Quote transferred into the pair: 380.999999999999999979
Attacker skim profit: 380.999999999999999979 quote
Pair reserves changed by the claim: 0
```

The attacker performed allocation, withdrawal for the pair and `skim` atomically. The proof used no role impersonation, direct Dividend funding, fabricated entitlement or storage modification.

The same underlying custody problem affects staking contracts, lending vaults, bridges and other pooled holders: rewards attach to the custody address rather than its beneficial owners. The V2 same-quote case is more severe because the reward is permissionlessly extractable instead of merely assigned to the custodian.

RECOMMENDATIONS:

Do not permit arbitrary callers to force claims for contract holders unless that holder has explicitly opted in to a receiver. Require holder authorization for receiver registration and claim initiation, or automatically exclude supported AMM and custody contracts and route their rewards through purpose-built distributors.

At minimum, monitor newly created pools and require each secondary custody address to be excluded before it receives launch tokens. Static launch-time defaults alone cannot cover future pools.

3.3 Tick-spacing and band updates can make registered quotes unlaunchable

SEVERITY: LOW **STATUS:** Fixed

PATH:

contracts/src/tax/TaxLaunchpadFactoryCore.sol

contracts/src/tax/TaxRWAERC20LaunchpadFactory.sol

DESCRIPTION:

setTickSpacing validates the current band offsets against the new spacing, but it does not revalidate any registered quote's start tick:

```
// TaxLaunchpadFactoryCore.sol:831-847
function _applyTickSpacing(int24 spacing) internal {
    if (spacing <= 0 || spacing > 16_384) revert OutOfBounds();
    for (uint256 i = 0; i < _bandTemplate.length; i++) {
        if (
            _bandTemplate[i].lowerOffset % spacing != 0
                || (_bandTemplate[i].upperOffset != TAIL && _bandTemplate[
   i].upperOffset % spacing != 0)
        ) revert MisalignedOffset();
    }
    emit TickSpacingUpdated(tickSpacing, spacing);
    tickSpacing = spacing;
}
```

The registered quote is checked only when a creator later launches:

```
// TaxRWAERC20LaunchpadFactory.sol:142-146, 189-192
function _beforeCreateLaunch(LaunchParams calldata launchParams) internal
   view override {
    ...
    _validateQuoteStartTick(_quoteConfig[launchParams.quoteToken].
   startTickToken0Frame);
}

function _validateQuoteStartTick(int24 startTickToken0Frame) internal view
    {
```

```
    _validateStartTickFrame(startTickToken0Frame);
    if (startTickToken0Frame % tickSpacing != 0) revert
   MisalignedStartTick();
}
```

Similarly, `setBandTemplate` validates relative offsets but not each registered quote's absolute resolved ticks. A template can pass its setter and later overflow a range, exceed TickMath bounds or resolve to zero liquidity for a particular quote during `_resolveSeedPositions`.

IMPACT:

An owner configuration transaction can succeed and bump `configVersion`, yet all launches for one or more already registered quotes can revert later. The failure is recoverable while an owner remains available, but it creates an avoidable operational outage and may be discovered only when a user attempts a launch.

RECOMMENDATIONS:

On tick-spacing or band-template changes, revalidate every registered quote against the complete resolved seed-position calculation, or require the owner to submit the affected sorted quote set and prove it matches the registry.

At minimum, deployment and management tooling must simulate one maximum-input launch for every registered quote after either global setting changes and before creation is enabled.

3.4 A nonpayable platform recipient can disable launches when the native launch fee is nonzero

SEVERITY: LOW **STATUS:** Fixed

PATH:

`contracts/src/tax/TaxLaunchpadFactoryCore.sol`

DESCRIPTION:

Native launch fees are pushed directly to the current platform recipient before launch creation completes:

```
// TaxLaunchpadFactoryCore.sol:452-458
function _payNativeLaunchFee(address payer) internal {
    uint256 amount = nativeLaunchFee;
    if (amount == 0) return;
    (bool success,) = payable(_feeState.platformRecipient).call{value:
    amount}('');
    if (!success) revert NativeLaunchFeeTransferFailed();
    emit NativeLaunchFeePaid(payer, _feeState.platformRecipient, amount);
}
```

Recipient validation rejects known internal helper addresses, but it does not and cannot prove that an arbitrary compatible contract accepts native transfers forever. A recipient may be nonpayable at configuration time or later upgrade to rejecting native value.

IMPACT:

When `nativeLaunchFee != 0`, every new `createLaunch` and `createLaunchAndBuy` reverts until the owner changes the platform recipient or sets the fee to zero. Existing launches and their accrued FeeEscrow credits remain unaffected.

RECOMMENDATIONS:

Accrue native launch fees through the existing pull-based escrow model instead of pushing them during creation. If push payment is retained, require an explicit payable compatibility rehearsal for the selected recipient and monitor any proxy implementation changes.

3.5 Native dividends trust an immutable wrapped-native dependency by address and code presence only

SEVERITY: LOW **STATUS:** Fixed

PATH:

`contracts/src/tax/TaxDividend.sol`

DESCRIPTION:

The Dividend implementation permanently records any nonzero contract as `wrappedNative`:

```
// TaxDividend.sol:98-102
constructor(address wrappedNativeAddress) {
    if (wrappedNativeAddress == address(0) || wrappedNativeAddress.code.
   length == 0) {
        revert InvalidConfiguration();
    }
    wrappedNative = wrappedNativeAddress;
    _disableInitializers();
}
```

For a native quote, pending allocation sends the clone's native balance to that dependency without measuring minted wrapped tokens:

```
// TaxDividend.sol:324-339
if (feeEscrow.owed(address(this), quoteToken) != 0) feeEscrow.claimFor(
   address(this), quoteToken);
if (quoteToken == address(0)) {
    uint256 nativeAmount = Math.min(address(this).balance, pending);
    if (nativeAmount != 0) ITaxDividendWrappedNative(wrappedNative).
   deposit{value: nativeAmount}();
}
uint256 balance = IERC20(rewardToken).balanceOf(address(this));
```

Claims similarly trust `withdraw(amount)` to return the expected native amount. There is no reciprocal binding, balance-delta assertion or code-identity check.

IMPACT:

An incorrect `deposit()` implementation can absorb native fees without minting wrapped tokens, leaving those fees pending and unallocatable. Already allocated liabilities become insolvent only if the dependency later destroys, seizes or rebases held backing, or refuses to redeem it. Because the dependency is embedded in the shared implementation, existing clones cannot be repaired by a factory setting.

RECOMMENDATIONS:

Verify the exact canonical wrapped-native address, runtime code and proxy status before deploying the Dividend implementation. Add balance-delta assertions around wrapping and unwrapping where compatible, and include a native round-trip in the deployment verifier. Treat any implementation change as a new suite requiring new clones.

3.6 Metadata is unbounded and unnormalized

SEVERITY: INFO **STATUS:** Fixed

PATH:

`contracts/src/tax/TaxLaunchToken.sol`

DESCRIPTION:

Token construction requires only nonempty name, symbol and metadata keys. It places no byte-length, item-count or character restrictions on names, symbols, URIs, keys or values:

```
// TaxLaunchToken.sol:56-81
if (
    genesisParams.factory == address(0) || bytes(genesisParams.tokenName).
    length == 0
        || bytes(genesisParams.tokenSymbol).length == 0 || genesisParams.
    initialRecipient == address(0)
        || genesisParams.initialSupply == 0
        || genesisParams.metadataKeys.length != genesisParams.
    metadataValues.length
) revert InvalidConfig();
...
for (uint256 i = 0; i < genesisParams.metadataKeys.length; i++) {
    string memory metadataKey = genesisParams.metadataKeys[i];
    if (bytes(metadataKey).length == 0) revert InvalidConfig();
    extraMetadata[metadataKey] = genesisParams.metadataValues[i];
}
```

Editable launches can later store unlimited additional keys and arbitrary new name or symbol bytes:

```
// TaxLaunchToken.sol:139-159
function updateName(string calldata newName) external onlyMetadata {
    if (bytes(newName).length == 0) revert InvalidConfig();
    _name = newName;
}

function updateExtraMetadata(string calldata metadataKey, string calldata
   metadataValue) external onlyMetadata {
    if (bytes(metadataKey).length == 0) revert InvalidConfig();
    extraMetadata[metadataKey] = metadataValue;
}
```

Control characters, bidirectional controls, zero-width characters and homoglyphs are accepted.

IMPACT:

A creator can produce extremely large metadata responses and event payloads or a name and symbol visually indistinguishable from another asset. Indexers, wallets and interfaces that do not independently bound and normalize this data may suffer resource exhaustion, layout corruption, log-processing problems or token impersonation.

Creation-time oversizing is partly self-limiting through transaction gas and initcode limits, but repeated post-launch updates can accumulate without an onchain count cap.

RECOMMENDATIONS:

Add conservative UTF-8 byte caps for names, symbols, URIs, keys and values, plus a maximum metadata-key count. Reject control and bidirectional formatting characters in name and symbol, or require offchain consumers to render escaped normalized text.

Interfaces must always use chain ID and token address as identity and must not treat a displayed name or symbol as unique or trusted.

3.7 Burn delivery is deferred, contrary to the supplied PR description

SEVERITY: INFO **STATUS:** Fixed

PATH:

`contracts/src/tax/TaxLaunchHook.sol`

`contracts/src/FeeEscrow.sol`

DESCRIPTION:

The supplied description says burn tokens go directly to the dead address during each successful trade and that burn-enabled sells require input to be settled before the swap.

The final Hook instead mints an ERC-6909 claim to FeeEscrow and records a fixed-sink credit:

```
// TaxLaunchHook.sol:497-503
if (burnFee != 0) {
    Currency tokenCurrency = key.currency1;
    if (cfg.tokenIsCurrency0) tokenCurrency = key.currency0;
    // Reserve claims without requiring physical seller input before the
   router settles.
    poolManager.mint(feeEscrow, tokenCurrency.toId(), burnFee);
    FeeEscrow(feeEscrow).credit(BURN_SINK, Currency.unwrap(tokenCurrency),
    burnFee);
    emit BurnTaxCredited(poolId, Currency.unwrap(tokenCurrency), BURN_SINK
   , burnFee);
}
```

Physical ERC20 delivery happens only when someone later calls FeeEscrow `claimFor(BURN_SINK, token)`. If nobody calls, delivery remains pending.

IMPACT:

`BurnTaxCredited` proves an irrevocably routed accounting allocation, not a contemporaneous dead-address token balance change. Analytics that count the event and the later `Claimed` or `Transfer` as separate burns will double count; analytics that infer physical delivery from the event will report it too early.

The fixed destination and fixed total supply prevent diversion, so this is a specification and integration mismatch rather than a fund-loss finding.

RECOMMENDATIONS:

Correct the PR description and every downstream indexer or UI specification to distinguish burn allocation, escrow redemption and physical dead-address delivery. If same-transaction physical delivery is a hard product requirement, the current code does not implement it.

3.8 The supplied validation snapshot is stale relative to the final source

SEVERITY: INFO **STATUS:** Fixed

PATH:

`docs/TAX_TOKEN_SUITE_VALIDATION.md`

DESCRIPTION:

The supplied handoff states 638 maintained tests plus 15 isolated cases, a 23,950-byte factory runtime and 42,736-byte maximum initcode. Those size values correspond to an earlier baseline, not the final claim-backed burn source at PR head.

Fresh compilation and the repository's final validation record agree on:

```
TaxRWAERC20LaunchpadFactory runtime: 23,850 bytes
Runtime headroom below EIP-170: 726 bytes
Maximum supported constructor initcode: 42,636 bytes
Initcode headroom below EIP-3860: 6,516 bytes
```

The current validation record also reports 656 maintained passing cases for the final candidate, while explicitly preserving historical test counts and stating which later changes did not rerun the entire matrix.

IMPACT:

Reviewers cannot reliably connect the supplied validation claims to the exact code being approved. The source is still within both size limits, but stale numbers weaken release assurance and can conceal whether a result belongs to the pre-change or final bytecode.

RECOMMENDATIONS:

Replace the handoff metrics with the exact final-source results and identify every result by commit, compiler settings, dependency revisions and whether it was freshly rerun or inherited from a source-identical checkpoint. Do not combine overlapping maintained and supplemental cases into one total.

The complete full/fork/invariant matrix should be rerun after fixing Findings 1 and 3 and after any change to production Solidity, generated bytecode or constructor inputs.

4. CONCLUSION

In this audit, we thoroughly analyzed **O1 Launch Tax Contracts** smart contract implementation. The problems found are described and explained in detail in Section 3. The problems found in the audit have been communicated to the project leader. We therefore consider the audit result to be **PASSED**.

To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.

5. APPENDIX

5.1 Basic Coding Assessment

5.1.1 Apply Verification Control

Description	The security of apply verification
Result	Not found
Severity	CRITICAL

5.1.2 Authorization Access Control

Description	Permission checks for external integral functions
Result	Not found
Severity	CRITICAL

5.1.3 Forged Transfer Vulnerability

Description	Assess whether there is a forged transfer notification vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.4 Transaction Rollback Attack

Description	Assess whether there is transaction rollback attack vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.5 Transaction Block Stuffing Attack

Description	Assess whether there is transaction blocking attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.6 Soft Fail Attack Assessment

Description	Assess whether there is soft fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.7 Hard Fail Attack Assessment

Description	Examine for hard fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.8 Abnormal Memo Assessment

Description	Assess whether there is abnormal memo vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.9 Abnormal Resource Consumption

Description	Examine whether abnormal resource consumption in contract processing
Result	Not found
Severity	CRITICAL

5.1.10 Random Number Security

Description	Examine whether the code uses insecure random number
Result	Not found
Severity	CRITICAL

5.2 Advanced Code Scrutiny

5.2.1 Cryptography Security

Description	Examine for weakness in cryptograph implementation
Result	Not found
Severity	HIGH

5.2.2 Account Permission Control

Description	Examine permission control issue in the contract
Result	Not found
Severity	MEDIUM

5.2.3 Malicious Code Behavior

Description	Examine whether sensitive behavior present in the code
Result	Not found
Severity	MEDIUM

5.2.4 Sensitive Information Disclosure

Description	Examine whether sensitive information disclosure issue present in the code
Result	Not found
Severity	MEDIUM

5.2.5 System API

Description	Examine whether system API application issue present in the code
Result	Not found
Severity	LOW

6. DISCLAIMER

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to the Company in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes without ExVul's prior written consent.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts ExVul to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. ExVul's position is that each company and individual are responsible for their own due diligence and continuous security. ExVul's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

7. REFERENCES

[1] MITRE. CWE-191: Integer Underflow (Wrap or Wraparound). https://cwe.mitre.org/data/definitions/191.html.

[2] MITRE. CWE-197: Numeric Truncation Error. https://cwe.mitre.org/data/definitions/197.html.

[3] MITRE. CWE-400: Uncontrolled Resource Consumption. https://cwe.mitre.org/data/definitions/400.html.

[4] MITRE. CWE-440: Expected Behavior Violation. https://cwe.mitre.org/data/definitions/440.html.

[5] MITRE. CWE-684: Protection Mechanism Failure. https://cwe.mitre.org/data/definitions/693.html.

[6] MITRE. CWE CATEGORY: 7PK - Security Features. https://cwe.mitre.org/data/definitions/254.html.

[7] MITRE. CWE CATEGORY: Behavioral Problems. https://cwe.mitre.org/data/definitions/438.html.

[8] MITRE. CWE CATEGORY: Numeric Errors. https://cwe.mitre.org/data/definitions/189.html.

[9] MITRE. CWE CATEGORY: Resource Management Errors. https://cwe.mitre.org/data/definitions/399.html.

[10] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology

8. About Exvul Security

Premier Security for the Web3 Ecosystem

ExVul is a premier Web3 security firm committed to forging a secure and trustworthy decentralized ecosystem. Our elite team consists of security veterans from world-leading technology and blockchain security firms, including Huawei, YBB Captical, Qihoo 360, Amber, ByteDance, MoveBit, and PeckShield. Team member Nolan is ranked as a top-40 whitehat on Immunefi and is the platform's sole All-Star in the APAC region.

Our expertise covers the full spectrum of Web3 security. We conduct **meticulous smart contract audits**, having fortified thousands of projects on chains like Evm, Solana, Aptos, Sui etc. Our **Blockchain Protocol Audits** secure the core infrastructure of L1/L2 by uncovering deep-seated vulnerabilities. We also offer **comprehensive wallet audits** to protect user assets and provide **proactive web3 pentest**, enabling partners to neutralize threats before they strike.

Trusted by industry leaders, ExVul is the security partner for **OKX, Bitget, Cobo, Infini, Stacks, Aptos, Sui, CoreDAO, Sei** etc.

EXVUL

Contact

Website
www.exvul.com

Email
contact@exvul.com

Twitter
@EXVULSEC

Github
github.com/EXVUL-Sec

ExVul